

Construção de Simulador PIC18

Guia Didático: Entendendo as Instruções ADDWF e INCF (Fluxo de Dados e Estouro)

Este documento destina-se a orientar a implementação simplificada de um simulador da arquitetura PIC18 voltado para o ensino. Para fins pedagógicos, **as flags de status do registrador STATUS foram omitidas**, permitindo que os alunos fiquem 100% focados na lógica do fluxo de dados, no tratamento de operandos e na limitação física de 8 bits.

1. Resumo das Operações

Instrução	Sintaxe / Opcode	Operação Lógica	Descrição Pedagógica
ADDWF	addwf f, d, a Opcode: 0010 01da ffff ffff	DEST = (W + f) & 0xFF	Soma o conteúdo do registrador de trabalho WREG com o registrador de memória f .
INCF	incf f, d, a Opcode: 0010 10da ffff ffff	DEST = (f + 1) & 0xFF	Incrementa em 1 o valor armazenado no registrador de memória f .

2. Parâmetros das Instruções (f , d , a)

f (File Register - 8 bits): Endereço do registrador na memória RAM/SFR (faixa de **0x00** a **0xFF**). Indica qual posição de memória participará da operação.

d (Destination - 1 bit): Define onde o resultado final será gravado:

- **d = 0** (ou **W**): O resultado é salvo no registrador de trabalho **WREG**.
- **d = 1** (ou **F**): O resultado é gravado de volta na memória no registrador **f**.

a (Access RAM - 1 bit): Seleciona o modo de desempenhamento do banco de memória:

- **a = 0** (Access Bank): Acessa diretamente a RAM rápida sem depender do registrador BSR.
- **a = 1** (Banked RAM): O endereço é composto combinando o registrador de banco (**BSR**) com o valor de **f**.

3. Largura de Bits e Tratamento do Estouro (Overflow)

Tanto o registrador **WREG** quanto cada posição de memória **f** possuem capacidade exata de **8 bits**, permitindo armazenar números inteiros de **0 a 255** (**0x00** a **0xFF**).

- **Estouro no ADDWF:** Se $W = 200$ (**0xC8**) e $f = 100$ (**0x64**), a soma matemática resulta em 300 . Como ultrapassa 255, ocorre o *roll-over* (estouro de 8 bits). O valor truncado fica $300 \bmod{256} = 44$ (**0x2C**).
- **Estouro no INCF:** Se $f = 255$ (**0xFF**), ao somar 1 , a soma atinge 256 . O valor truncado em 8 bits **retorna para 0** (**0x00**).

- **Aplicação em Python:** Utiliza-se a máscara bitwise `& 0xFF` logo após a operação aritmética para descartar automaticamente qualquer bit acima do 8º bit.

EXEMPLO DE AULA Implementação da Instrução ADDWF em Python

Abaixo está o código de referência para a instrução **ADDWF**, estruturado para o simulador didático:

```
# ---- ADDWF: Soma W com f -----
# Opcode PIC18: 0010 01da ffff ffff (Mascara 0xFC00 == 0x2400)
if (op & 0xFC00) == 0x2400:
    val_w = self.W
    val_f = self.mem.ler(f, acesso)

    # 1. Realiza a soma e força o limite de 8 bits (máscara 0xFF)
    resultado = (val_w + val_f) & 0xFF

    # 2. Desvia o destino conforme o bit d (0 -> W, 1 -> f)
    if d:
        self.mem.escrever(f, resultado, acesso)
    else:
        self.W = resultado

    return f"addwf 0x{f:02X}, {'f' if d else 'w'}, {acesso}", 1
```

EXERCÍCIO PRÁTICO Implementação da Instrução INCF

Objetivo: Com base no exemplo da instrução **ADDWF** demonstrado acima, escreva o trecho de código em Python para a instrução **INCF** (Increment f).

Requisitos:

1. Verificar a máscara do opcode de **INCF** (`(op & 0xFC00) == 0x2800`).
2. Ler o valor do registrador **f** usando `self.mem.ler(f, acesso)` .
3. Incrementar o valor em 1 e garantir o truncamento em 8 bits (`& 0xFF`) para tratar o estouro de `0xFF` para `0x00` .
4. Salvar o resultado em **f** ou em **w** dependendo do valor do bit **d** .

Espaço para o aluno escrever o código da função INCF:

```
if (op & 0xFC00) == 0x2800:
    # 1. Ler registrador f

    # 2. Calcular incremento com máscara de 8 bits

    # 3. Armazenar no destino correto (d)
```